



RTL LABORATORY FILE

SUBMITTED BY:

NAME	ABDUL RAFAY
------	-------------

SUBMITTED TO:

SIR KHALIL REHMAN

(RTL) WEEK # 04 LAB

Table Of Contents

AHB-LITE	-----3
----------	--------

RTL LAB WEEK # 04

DESIGN AND SIMULATION OF AHB-LITE SLAVE INTERFACE USING SYSTEMVERILOG

➤ OBJECTIVE:

To design and simulate an AHB-Lite Slave interface capable of handling memory read and write operations using the AHB-Lite bus protocol.

LAB TASKS

➤ Question 1: Design and simulate a AHB-Lite using SystemVerilog.

❖ CODE:

• DESIGN CODE:

```
// ABDUL RAFAY
// PSEB USTP INSPIRE SESSION
// SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
// WEEK # 02 (RTL)
// AHB SLAVE
module ahb_slave #(
    parameter ADDR_WIDTH = 32,
    parameter DATA_WIDTH = 32,
    parameter MEM_DEPTH = 8
)(
    input logic          HCLK,
    input logic          HRESETn,
    input logic          HSEL,
    input logic [ADDR_WIDTH-1:0] HADDR,
    input logic          HWRITE,
    input logic [1:0]      HTRANS,
    input logic [2:0]      HSIZE,
    input logic [DATA_WIDTH-1:0] HWDATA,
    input logic           HREADY,
    output logic [DATA_WIDTH-1:0] HRDATA,
    output logic          HREADYOUT,
    output logic          HRESP
);
    localparam TRN_IDLE = 2'b00;
    localparam TRN_NONSEQ = 2'b10;
    logic [DATA_WIDTH-1:0] mem [0:MEM_DEPTH-1];
    logic [ADDR_WIDTH-1:0] addr_reg;
    logic          write_reg;
    logic          active_reg;
    always_ff @(posedge HCLK or negedge HRESETn) begin
        if (!HRESETn) begin
            addr_reg <= '0;
```

```

        write_reg <= '0;
        active_reg <= '0;
    end else if (HREADY) begin
        if (HSEL && (HTRANS == TRN_NONSEQ)) begin
            addr_reg <= HADDR;
            write_reg <= HWRITE;
            active_reg <= 1'b1;
        end else begin
            active_reg <= 1'b0;
        end
    end
end
always_comb begin
    HREADYOUT = 1'b1;
    HRESP     = 1'b0;
end
always_ff @(posedge HCLK or negedge HRESETn) begin
    if (!HRESETn) begin
        HRDATA <= '0;
        for (int i = 0; i < MEM_DEPTH; i++) begin
            mem[i] <= '0;
        end
    end else begin
        if (active_reg && HREADYOUT) begin
            if (write_reg) begin
                mem[addr_reg % MEM_DEPTH] <= HWDATA;
            end else begin
                HRDATA <= mem[addr_reg % MEM_DEPTH];
            end
        end
    end
end
end
endmodule

```

• TEST BENCH CODE:

```

// // ABDUL RAFAY
// PSEB USTP INSPIRE SESSION
// SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
// WEEK # 02 (RTL)
// TEST BENCH FOR AHB SLAVE
`timescale 1ns / 1ps
module tb_ahb_slave;
    localparam ADDR_WIDTH = 32;
    localparam DATA_WIDTH = 32;
    localparam CLK_PERIOD = 10;
    logic      HCLK;
    logic      HRESETn;
    logic      HSEL;
    logic [ADDR_WIDTH-1:0] HADDR;
    logic      HWRITE;

```

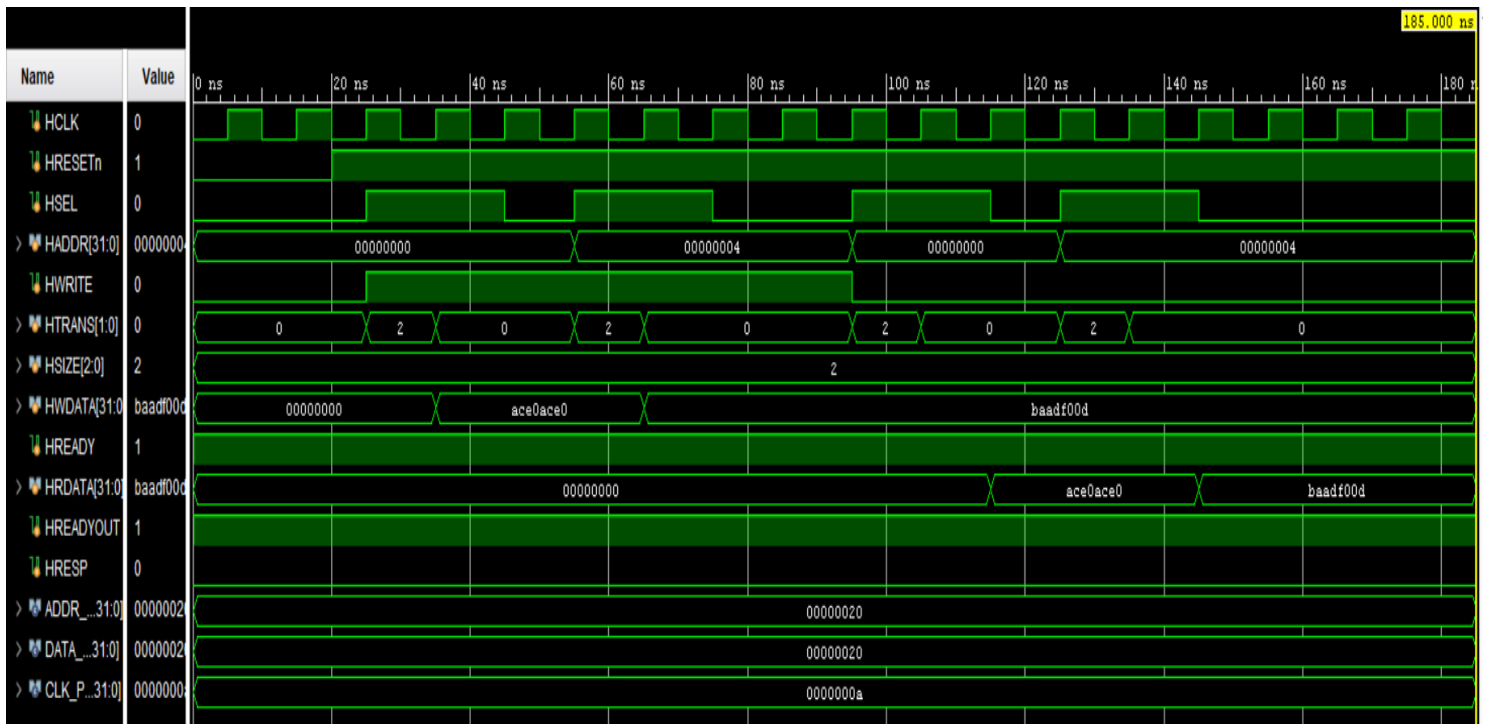
```

logic [1:0]      HTRANS;
logic [2:0]      HSIZE;
logic [DATA_WIDTH-1:0] HWDATA;
logic           HREADY;
logic [DATA_WIDTH-1:0] HRDATA;
logic           HREADYOUT;
logic           HRESP;
ahb_slave #(
    .ADDR_WIDTH(ADDR_WIDTH),
    .DATA_WIDTH(DATA_WIDTH),
    .MEM_DEPTH(8)
) dut (
    .HCLK(HCLK),
    .HRESETn(HRESETn),
    .HSEL(HSEL),
    .HADDR(HADDR),
    .HWRITE(HWRITE),
    .HTRANS(HTRANS),
    .HSIZE(HSIZE),
    .HWDATA(HWDATA),
    .HREADY(HREADY),
    .HRDATA(HRDATA),
    .HREADYOUT(HREADYOUT),
    .HRESP(HRESP)
);
always #(CLK_PERIOD / 2) HCLK = ~HCLK;
task ahb_write(input logic [ADDR_WIDTH-1:0] addr, input logic [DATA_WIDTH-1:0] data);
    @(posedge HCLK);
    HSEL <= 1'b1;
    HADDR <= addr;
    HWRITE <= 1'b1;
    HTRANS <= 2'b10;
    @(posedge HCLK);
    HTRANS <= 2'b00;
    HWDATA <= data;
    @(posedge HCLK);
    HSEL <= 1'b0;
endtask
task ahb_read(input logic [ADDR_WIDTH-1:0] addr);
    @(posedge HCLK);
    HSEL <= 1'b1;
    HADDR <= addr;
    HWRITE <= 1'b0;
    HTRANS <= 2'b10;
    @(posedge HCLK);
    HTRANS <= 2'b00;
    @(posedge HCLK);
    HSEL <= 1'b0;
endtask
initial begin
    HCLK = 0;

```

```
HRESETn = 0;
HSEL  = 0;
HADDR = 0;
HWRITE = 0;
HTRANS = 2'b00;
HSIZE = 3'b010;
HWDATA = 0;
HREADY = 1;
#20;
HRESETn = 1;
$display("[TB] Reset Deasserted. Starting AHB Transfers...");
ahb_write(32'h0000_0000, 32'hACE0_ACE0);
ahb_write(32'h0000_0004, 32'hBAAD_F00D);
#20;
ahb_read(32'h0000_0000);
ahb_read(32'h0000_0004);
#40;
$display("[TB] AHB Simulation Completed Successfully!");
$finish;
end
endmodule
```

❖ WAVE FORM OUTPUT:



❖ SCHEMATIC OUTPUT:

